

arm

Formal Verification

Lecture for ETIN40

Part 1

Lars Viklund

2026-10-05

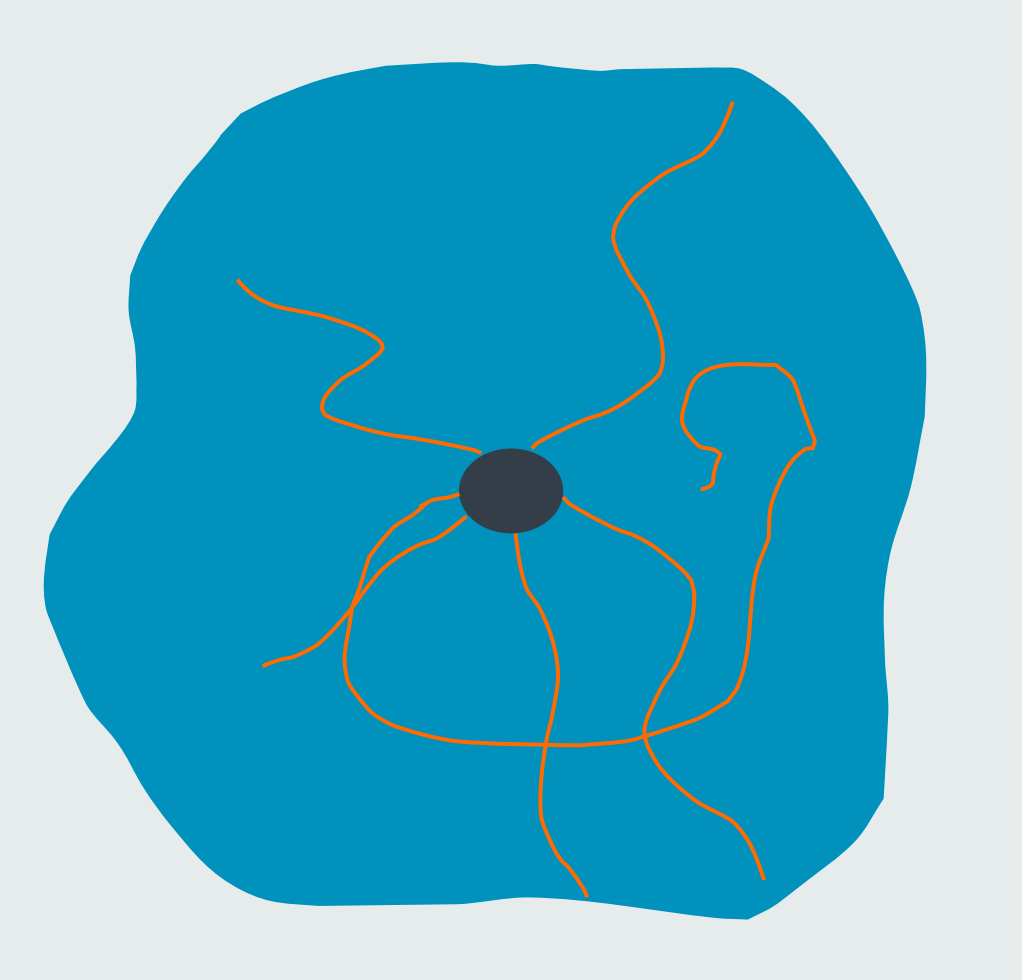
© 2026 Arm

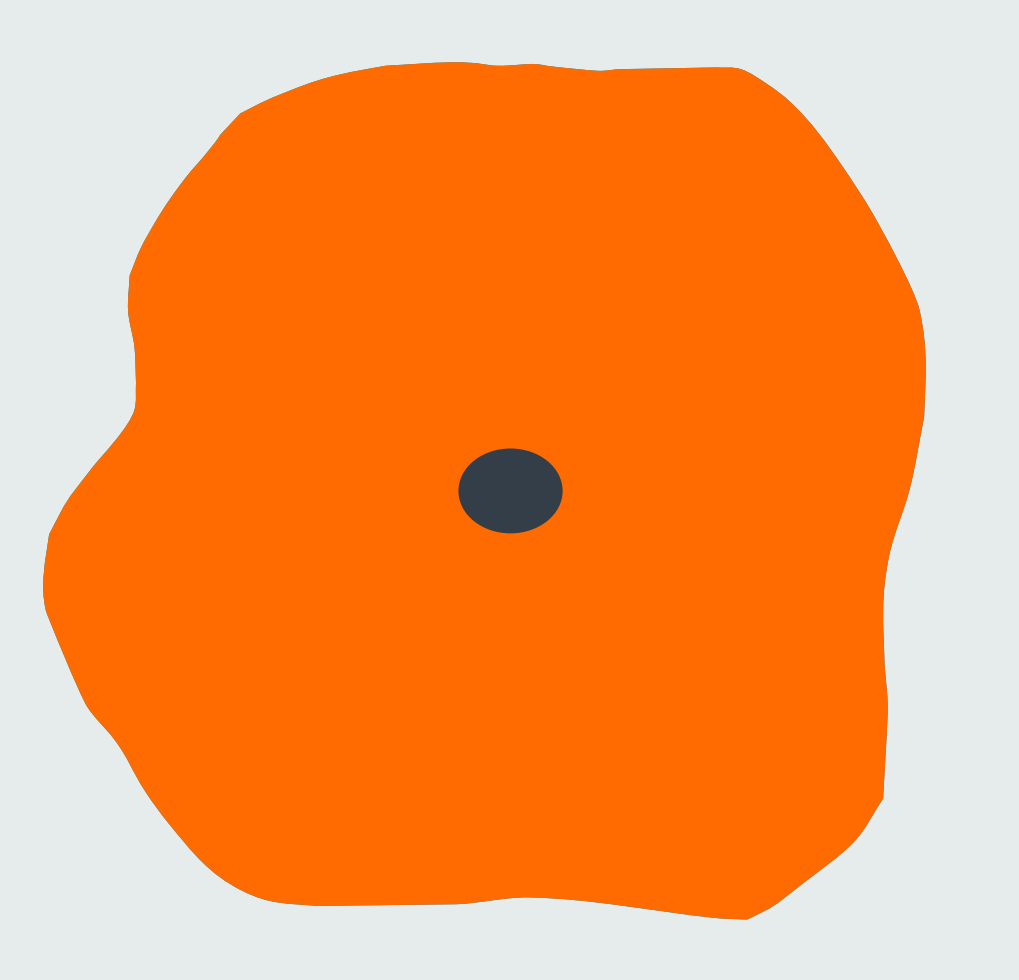
What is Formal Verification?

- Use of tools to analyze possible design behaviors
- Differs from simulation, which evaluates specific input sequences
- Focus: formal verification of hardware

© 2026 Arm

Simulation vs. Formal





© 2026 Arm

Advantages of Formal Verification

- Exhaustive Analysis
  - All possible design behaviors analyzed
- Corner Cases
  - Likely to uncover interesting corner cases
- Minimal Examples
  - Tools typically generate minimal examples
- State-Driven and Output-Driven Analysis
  - Allows constraining the behaviors of the design at any point
- Understanding Infinite Behaviors
  - Allows asking and answering questions about design behavior over unbounded time periods

© 2026 Arm

General Use Models for Formal Property Verification (FPV)

- Design Exploration FPV
  - Bring-up of new RTL before testbenches are available
  - Understanding legacy RTL
- Bug Hunting FPV
  - Extra level of confidence for designs with many corner cases
  - Specific features known to be error-prone and hard to verify
- Signoff Quality FPV
  - Full proofs
- Application-specific FPV
  - Formal apps for specific problems

© 2026 Arm

Design Exploration FPV

- Bring-up of new RTL to see how it behaves in simple cases
- Early verification before testbenches are ready
- Explore how a partial design behaves under known assumptions
- Understanding typical behaviors of legacy RTL code
- Exploring the suitability of FPV on a new RTL design

© 2026 Arm

Bug Hunting FPV

- Extra level of confidence for designs with many corner cases
- Specific features known to be error-prone and hard to verify
- Can build on design exploration FPV

© 2026 Arm

Signoff Quality FPV

- Full proofs, or
- Potentially bounded proofs with justified depth
- Metrics (formal coverage)

© 2026 Arm

Formal Verification Usage Levels

1. Auto Formal
   - auto checks
   - X-propagation
   - unreachability
2. Formal Apps
   - connectivity checking
   - register checks
   - clock gating verification
3. Assertion Based Verification (ABV)
   - RTL assertions
   - protocol checks
4. Block Signoff
5. System Architecture Verification

© 2026 Arm

Model Checking

- Used by tools to check if a sequential system meets a given specification
- Inputs:
  - Specification
    - E.g., SVA properties
  - Model
    - E.g., RTL code
- The specification is typically more abstract than the model
- Searches the model's possible future states for specification violations
- Two main techniques:
  - Binary Decision Diagrams (BDD)
  - Satisfiability (SAT)

© 2026 Arm

SystemVerilog Assertion (SVA)

- Assertions – Statements about the design that are expected to always be true
- Assumptions – Conditions that typically constrain the environment
  - In simulation, assumptions are treated the same as assertions
  - In FPV, tools assume these conditions are true
- Cover properties – Describe scenarios we want to see happen
- Assertions are proof targets in FPV
  - Proved – Assertion holds
  - Disproved – Counterexample (CEX) found
  - Inconclusive – Neither proved nor disproved
- The term *assertion statement* can refer to assertions, assumptions, or cover properties

© 2026 Arm

Assertion Statements

- Immediate
  - Checked when execution reaches the assertion
- Concurrent
  - Evaluated relative to one or more clocks
  - Can describe behavior that spans time

© 2026 Arm

Concurrent Assertions

- Use sampled values
- Require a clock expression
- Allow specification of reset conditions

© 2026 Arm

SVA Layers

1. Booleans
   - SystemVerilog Boolean expressions
2. Sequences
   - Statements about Booleans (or other sequences) over time
3. Properties
   - Combine sequences with additional operators to describe behavior
4. Assertion statements
   - Statements using one of the keywords assert, assume, or cover

© 2026 Arm

Sequences

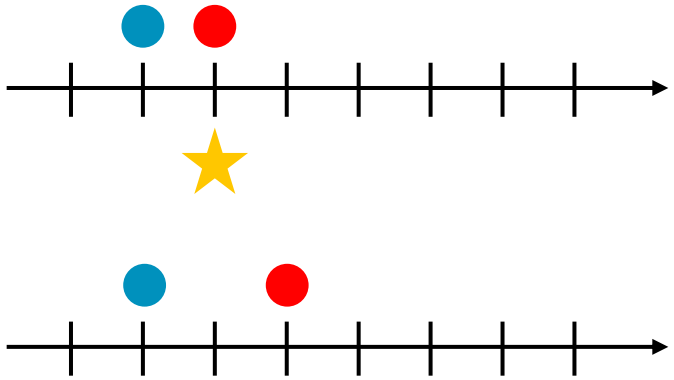
- Describe sequential behaviors
- Sequence operators:
  - Delay specifiers
  - Repetition
    - Consecutive
    - Goto (nonconsecutive exact)
    - Nonconsecutive
  - Other operators
    - `and`
    - `or`
    - `throughout`
    - `within`
    - `intersect`
    - …

© 2026 Arm

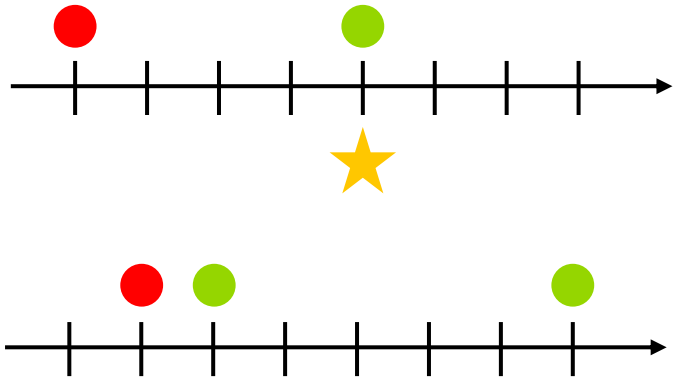
Delays

- ##*N*
- ##[*A*:*B*]
- ##[+] (equivalent to ##[1:$])
- ##[*] (equivalent to ##[0:$])

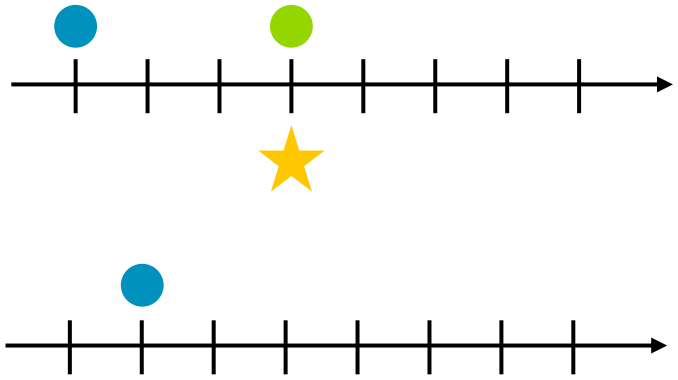
`blue ##1 red`



`red ##[2:4] green`



`blue ##[+] green`

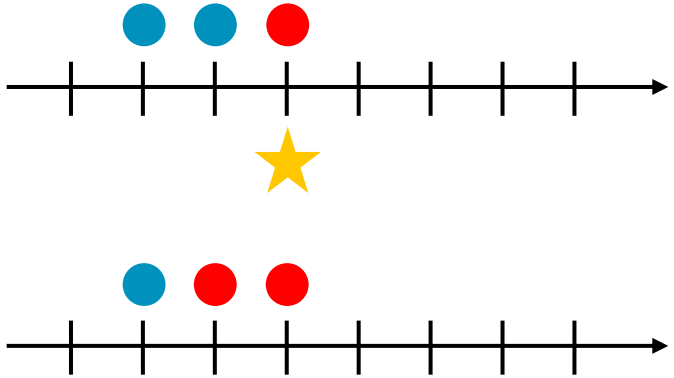


© 2026 Arm

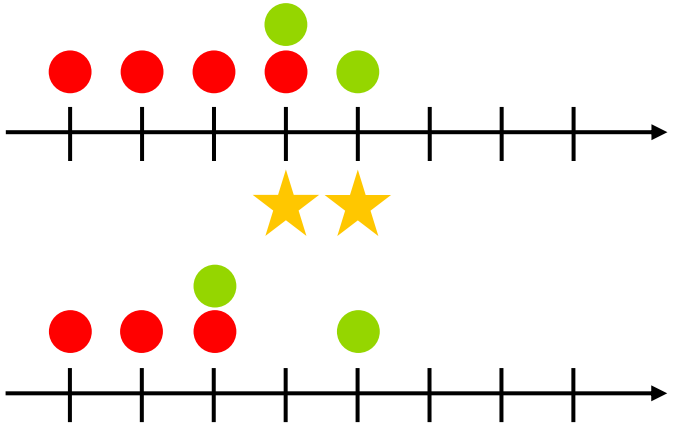
Consecutive Repetition

- `[*N]`
- `[*A:B]`
- `[+]` (equivalent to `[*1:$]`)
- `[*]` (equivalent to `[*0:$]`)

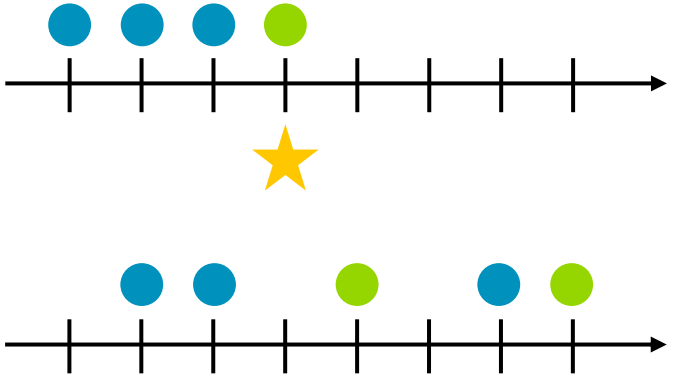
`blue [*2] ##1 red`



`red [*3:4] ##1 green`



`blue [*2:$] ##1 green`

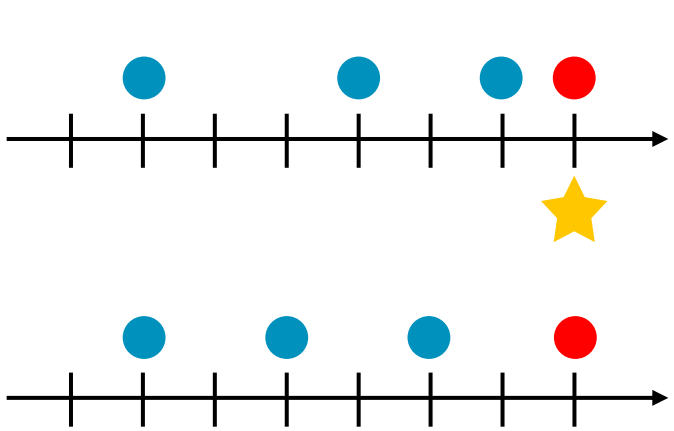


© 2026 Arm

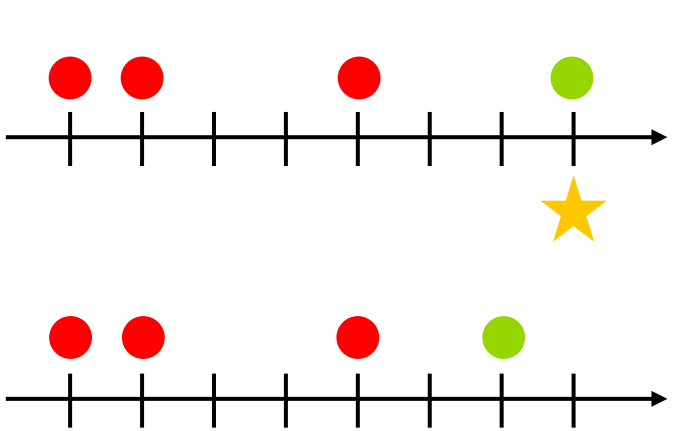
Goto Repetition

- `[->N]`
- `[->A:B]`

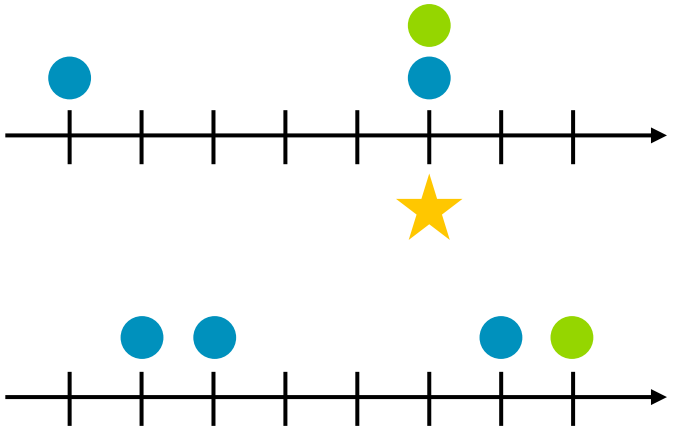
`blue [->3] ##1 red`



`red [->2:3] ##3 green`



`blue [->2] ##0 green`

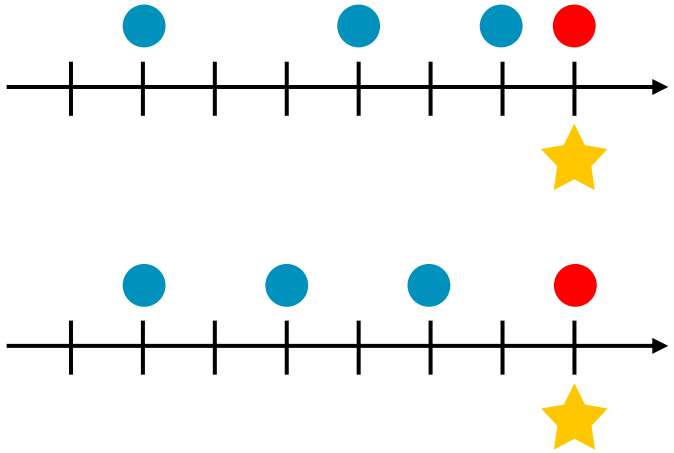


© 2026 Arm

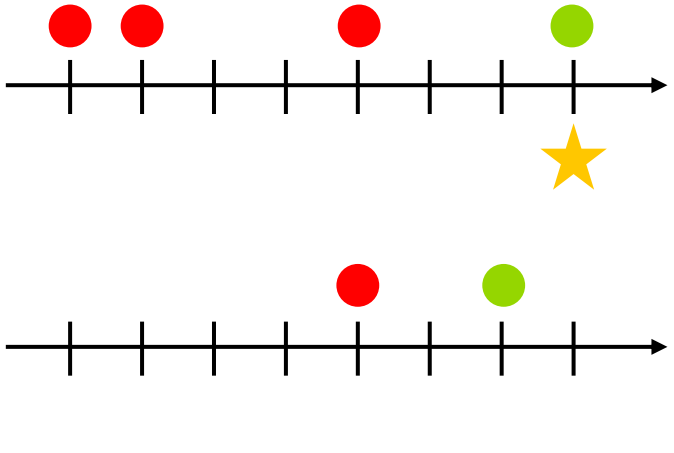
Nonconsecutive Repetition

- `[=N]`
- `[=A:B]`

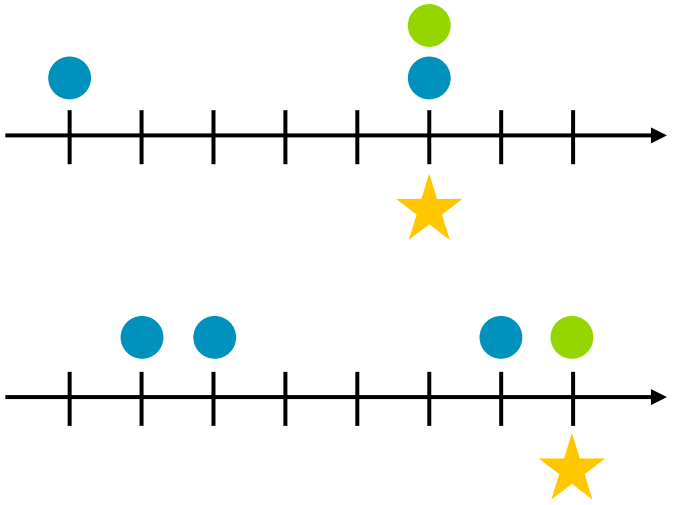
`blue [=3] ##1 red`



`red [=2:3] ##1 green`



`blue [=2] ##0 green`

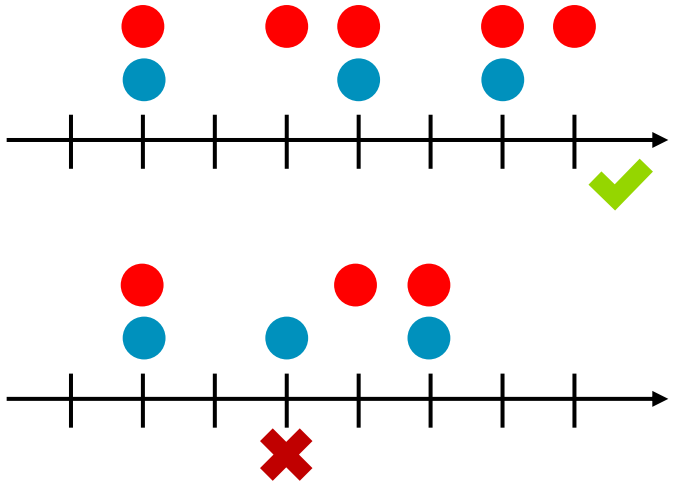


© 2026 Arm

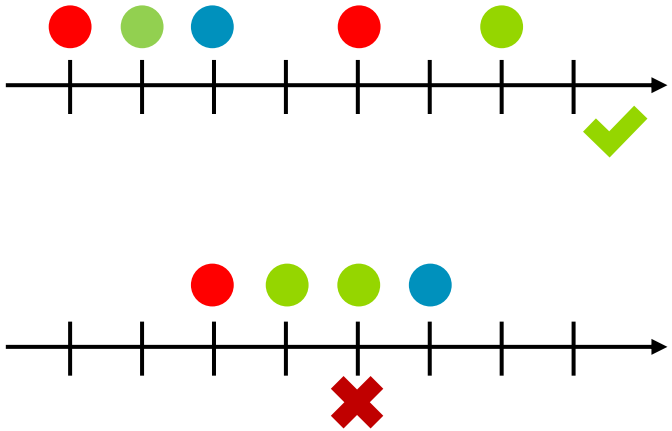
Properties

- Triggered implication constructs
  - *sequence* `|->` *property*
  - *sequence* `|=>` *property*
- Left-hand side: *antecedent*
- Right-hand side: *consequent*

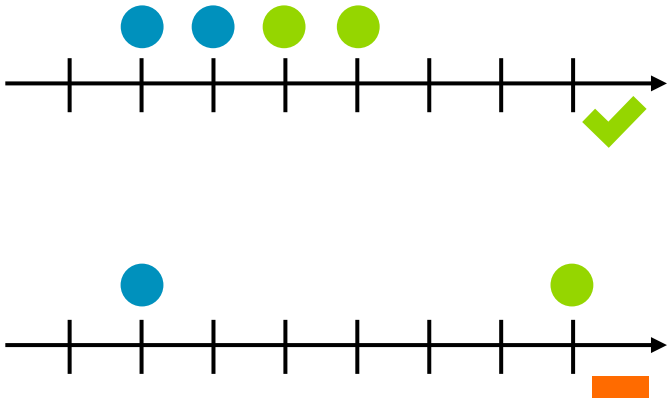
`blue |-> red`



`red ##1 green |=> blue`



`blue [*2] |=> green [*2]`

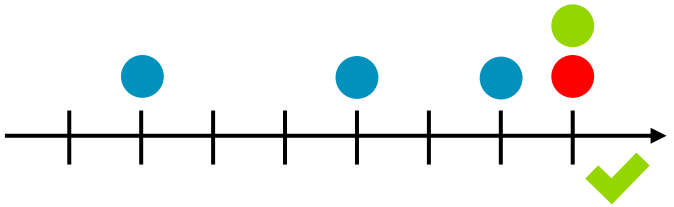


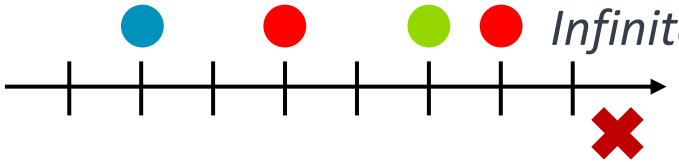
© 2026 Arm

Linear Temporal Logic Operators

- Express liveness properties
  - `s_eventually`

```
blue |-> s_eventually(red && green)
```





Infinite loop with alternating red and green

© 2026 Arm
arm

SVA Tips

- Use concurrent assertions instead of immediate assertions when possible
- Use *final deferred* immediate assertions for combinational checks
- Use triggered properties when possible
  - `a_boolean: assert property (!a || b);`
  - `a_trigger: assert property (a |-> b);`
- Make each assertion as simple as possible
- Write simple properties and use auxiliary SystemVerilog code when needed
- Use named sequences and properties for more readable code
- Use `bind` statements to separate verification assertions from RTL
- Use default clocking and default `disable iff` statements
- Use a structured naming of properties to facilitate changing some to assumptions

© 2026 Arm

arm

Thank You
Danke
Gracias
Grazie
谢谢
ありがとう
Asante
Merci
감사합니다
धन्यवाद
Kiitos
شكرًا
ধন্যবাদ
תודה
ధన్యవాదములు

© 2026 Arm



The Arm trademarks featured in this presentation are registered trademarks or trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All rights reserved. All other marks featured may be trademarks of their respective owners.

www.arm.com/company/policies/trademarks

© 2026 Arm